Informe 2

Autores:

Eric Guacache C.I: 31.788.759
Renny Andrade C.I: 29.727.828

Índice

1. Diferencias entre Registros Temporales ($t0–$t9) y Registros Guardados ($s0–$s7) 2
1.1. Diferencias Conceptuales 2
1.2. Aplicación en los Algoritmos (Quicksort y Mergesort) 2
1.3. Justificación del Diseño en Algoritmos Recursivos 3

2. Diferencias entre los Registros $a0–$a3, $v0–$v1 y $ra 3

3. Impacto de Registros frente a Memoria en el Rendimiento 4

4. Impacto de Estructuras de Control y Saltos 4

5. Diferencias de Complejidad Computacional e Implicaciones 5
5.1. Implicaciones para la Implementación en MIPS32 5
5.2. Comparación Práctica y Recomendaciones 6

6. Fases del Ciclo de Ejecución en MIPS32 (Camino de Datos) 6

7. Predominio de Tipos de Instrucciones (R, I, J) 7

8. Impacto del Abuso de Instrucciones de Salto 7

9. Ventajas del Modelo RISC de MIPS 8

10. Uso del Modo Paso a Paso (Step / Step Into) en MARS 8

11. Herramienta de MARS para Inspección de Registros 8

12. Visualización del Camino de Datos para Instrucción Tipo R (add) 9

13. Visualización del Camino de Datos para Instrucción Tipo I (lw) 9

14. Justificación de los Algoritmos Seleccionados 9

15. Análisis y Discusión de los Resultados 10

1. Diferencias entre Registros Temporales ($t0–$t9) y Registros Guardados ($s0–$s7)

1.1. Diferencias Conceptuales

En la arquitectura MIPS32, los registros se dividen en dos categorías principales con convenciones de uso específicas:

- **Registros Temporales ($t0–$t9):**
  - **Propósito:** Almacenamiento temporal de valores intermedios durante cálculos puntuales.
  - **Responsabilidad:** El llamante (*caller*) debe asumir que pueden modificarse tras una llamada a función.
  - **Preservación:** No se garantiza que conserven su valor después de un `jal`.
  - **Uso típico:** Variables auxiliares en cálculos aritméticos, índices locales dentro de bucles de una sola subrutina.
- **Registros Guardados ($s0–$s7):**
  - **Propósito:** Valores que deben persistir a través de llamadas a subrutinas.
  - **Responsabilidad:** El llamado (*callee*) debe preservarlos en la pila (*stack*) antes de modificarlos y restaurarlos antes de retornar.
  - **Preservación:** Deben mantener su valor antes y después de cualquier `jal`.
  - **Uso típico:** Punteros a arreglos, índices de límites (*left*, *right*, *mid*) o variables críticas que se utilizan a lo largo de llamadas recursivas o anidadas.

1.2. Aplicación en los Algoritmos (Quicksort y Mergesort)

Dado que tanto Quicksort y Mergesort se basan en la técnica de divide y vencerás y emplean recursividad, el uso de registros guardados (`$s0`–`$s7`) y del stack se vuelve fundamental en ambas implementaciones:

- **En Mergesort:** Se utilizan registros guardados (como `$s0`, `$s1` y `$s2`) para preservar de manera segura el puntero base del arreglo, el índice inicial (*left*) y el índice final (*right*), así como el punto medio (*mid*) calculado. Al realizar las dos llamadas recursivas sucesivas (`jal mergesort` para la primera mitad y `jal mergesort` para la segunda mitad), estos valores en los registros `$s` deben permanecer intactos para calcular correctamente los rangos del paso posterior de combinación (`jal merge`).
- **En Quicksort:** De forma análoga, registros como `$s0` y `$s1` almacenan los límites de la partición actual. Se requiere asegurar su valor en la pila antes de ejecutar la partición (`jal partition`) y la llamada recursiva al subarreglo izquierdo.

1.3. Justificación del Diseño en Algoritmos Recursivos

- **Persistencia tras invocaciones:** A diferencia de algoritmos iterativos simples, el flujo de ejecución recursivo con `jal` sobrescribe rutinariamente los registros temporales. Por ello, delegar variables de estado clave a `$s0`–`$s7` previene la corrupción de datos al regresar de una llamada recursiva.
- **Garantía del convenio ABI:** Guardar los registros `$s` y el registro de retorno `$ra` en la pila al inicio de la subrutina (prólogo) y restaurarlos al final (epílogo) garantiza la corrección del programa en entornos MIPS32.
- **Uso puntual de temporales:** Los registros `$t0`–`$t9` se reservan únicamente dentro de las rutinas auxiliares de bajo nivel (como `merge` o `partition`) para comparaciones de elementos e incrementos de índices temporales donde no se realizan nuevas llamadas anidadas.

2. Diferencias entre los Registros $a0–$a3, $v0–$v1 y $ra

- **Registros de Argumentos ($a0–$a3):**
  - **Propósito:** Transferir parámetros a las subrutinas.
  - **Aplicación práctica:** En Quicksort, se usan `$a0` (dirección del arreglo), `$a1` (*low*) y `$a2` (*high*). En Mergesort, se utiliza adicionalmente `$a3` para pasar el punto medio (*mid*) al invocar a la rutina de combinación `merge` (`jal merge`).
- **Registros de Retorno ($v0–$v1):**
  - **Propósito:** Devolver valores de las funciones y seleccionar códigos de servicios del sistema (`syscall`).
  - **Aplicación práctica:** La función `partition` en Quicksort calcula la posición definitiva del pivote y la retorna a través de `$v0` (`move $v0, $t2`). En los bloques de impresión y llamadas del sistema, se carga en `$v0` el servicio solicitado (`li $v0, 4` para strings, `li $v0, 1` para enteros, `li $v0, 10` para finalizar).
- **Registro de Dirección de Retorno ($ra):**
  - **Propósito:** Almacenar la dirección a la que debe volver la ejecución tras un salto `jal`.
  - **Aplicación práctica:** Debido a que tanto Mergesort como Quicksort realizan llamadas recursivas (múltiples `jal`), el valor de `$ra` se sobrescribe en cada llamada anidada. Por esta razón, ambas funciones reservan espacio en el stack y salvan `$ra` explícitamente (`sw $ra, 16($sp)`) antes de llamar a subrutinas y lo restauran (`lw $ra, 16($sp)`) antes de ejecutar `jr $ra`.

3. Impacto de Registros frente a Memoria en el Rendimiento

En MIPS32, las operaciones entre registros se ejecutan en un único ciclo de reloj dentro de la ALU, mientras que las instrucciones de acceso a memoria (`lw`/`sw`) requieren acceder al subsistema de datos, aumentando la latencia y los posibles estancamientos del pipeline.

Cuadro 1: Tipos de acceso en cada algoritmo

Operación	**Mergesort**	**Quicksort**
Accesos a Registros	Alto (cálculo de rangos/índices)	Alto (comparaciones e incrementos)
Accesos a Memoria (E/S)	Muy Alto (copia constante a `temp`)	Medio-Bajo (*swaps* in-place)
Uso de Stack (Pila)	Alto (20 bytes por marco)	Alto (20 bytes por marco)

- **Caso Mergesort:** Presenta un alto costo de memoria secundaria. Durante la etapa `merge`, lee los elementos de `array`, los escribe en un arreglo auxiliar `.space 100` (`temp`), y luego ejecuta el bucle `loop_copia` para volcar de nuevo los datos a la memoria original. A pesar de apoyarse en registros temporales (`$t0`–`$t2`) para controlar los índices i, j, k, la gran cantidad de lecturas (`lw $t4, 0($t3)`) y escrituras (`sw $t5, 0($t3)`) genera un cuello de botella por accesos a memoria de datos.
- **Caso Quicksort:** Optimiza el acceso a memoria al operar *in-place*. En la rutina `partition`, los elementos se comparan directamente en registros (`lw $t5, 0($t4)`) y solo se modifican en memoria cuando ocurre un intercambio (*swap*). No requiere un búfer temporal auxiliar.

4. Impacto de Estructuras de Control y Saltos

Las bifurcaciones condicionales (`bge`, `ble`, `bgt`) provocan penalizaciones por salto (*branch penalty*) en el pipeline cuando la predicción de salto falla.

- **En Mergesort:** Los bucles como `loop_comparar` evalúan bordes fijos (`bgt $t0, $a3` y `bgt $t1, $a2`). Dado que los límites del subarreglo son predecibles, el predictor de saltos del procesador funciona de forma eficiente. Sin embargo, la estructura recursiva doble (`jal mergesort`) impone una alta carga fija por la manipulación continua de punteros de la pila (`addi $sp, $sp, -20`).
- **En Quicksort:** La condición del bucle `part_loop` depende del valor de los datos comparados contra el pivote (`bge $t5, $t1, part_next`). Dependiendo de cuán desordenado esté el arreglo original, este salto puede volverse altamente impredecible, forzando la limpieza (*flush*) del pipeline repetidamente.

- **Optimizaciones:** Reorganizar instrucciones dentro de las ranuras de retardo (*delay slots*) tras las instrucciones `j` o `jal` e implementar desenrollado de bucles en las etapas de copia o intercambio de elementos.

5. Diferencias de Complejidad Computacional e Implicaciones

Cuadro 2: Complejidad computacional comparada

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio auxiliar
Quicksort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$ (pila)
Mergesort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$ (pila + `temp`)

5.1. Implicaciones para la Implementación en MIPS32

- **Mergesort:**
  - *Garantía de Rendimiento:* Ofrece un tiempo de ejecución acotado a $\mathcal{O}(n \log n)$ incluso en el peor de los casos, lo que evita degeneraciones de tiempo en la CPU.
  - *Impacto en Memoria:* Requiere reservar un arreglo auxiliar explícito en la sección `.data` (`temp: .space 100`). En ensamblador MIPS32, transferir continuamente los datos de `array` a `temp` y copiar el resultado ordenado de vuelta implica una cantidad masiva de instrucciones `lw` y `sw` dentro del bucle `copiar_a_original`, sobrecargando el bus de memoria de datos.
  - *Uso de Pila:* Genera marcos de pila uniformes de 20 bytes (`addi $sp, $sp, -20`) para almacenar `$ra`, `$s0` (el punto medio *mid*) y los argumentos.
- **Quicksort:**
  - *Sensibilidad a los Datos:* Su eficiencia depende de la elección del pivote. Si el arreglo ya está ordenado y se toma el último elemento como pivote, el árbol de recursión se degrada a profundidad $\mathcal{O}(n)$, cayendo a una complejidad cuadrática $\mathcal{O}(n^2)$.
  - *Eficiencia In-Place:* No utiliza memoria auxiliar externa para los elementos. Todas las reordenaciones se realizan intercambiando registros mediante punteros directos en la rutina `partition`. Esto minimiza drásticamente los accesos al subsistema de memoria principal en comparación con Mergesort.
  - *Uso de Pila:* Al igual que Mergesort, reserva 20 bytes por llamada recursiva para preservar `$ra`, `$s0` (índice *pi*) y los rangos *low/high*.

Cuadro 3: Impacto práctico en la implementación MIPS32

Factor	**Quicksort**	**Mergesort**
Instrucciones	Menor cantidad (uso *in-place*)	Mayor cantidad (bucle de copia `merge`)
Tráfico Memoria	Moderado (solo escrituras en *swaps*)	Muy alto (lecturas/escrituras en `temp`)
Predictibilidad	Variable (depende del pivote)	Alta (condiciones fijas i, j, k)
Memoria RAM	Arreglo original + pila	Arreglo original + Buffer `temp`

5.2. Comparación Práctica y Recomendaciones

- **Priorizar Quicksort para sistemas restringidos:** Dado que MIPS32 es común en sistemas embebidos, la menor huella de memoria de Quicksort (*in-place*) reduce la presión sobre el caché de datos y ahorra memoria principal.
- **Optimización del pivote:** Implementar la técnica de la mediana de tres para elegir el pivote garantizaría mantener la complejidad $\mathcal{O}(n \log n)$ sin caer en el peor caso.
- **Optimización de Mergesort:** Para reducir el *overhead*, la etapa `copiar_a_original` puede optimizarse intercalando punteros o utilizando asignaciones dinámicas que eviten el volcado constante del buffer.

6. Fases del Ciclo de Ejecución en MIPS32 (Camino de Datos)

1. **Búsqueda de la Instrucción (IF - Instruction Fetch):** Se utiliza el Contador de Programa (PC) para leer la dirección de la siguiente instrucción desde la memoria de instrucciones. La instrucción leída se almacena en el registro de instrucción (IR) y el puntero se actualiza: $PC \leftarrow PC + 4$.
2. **Decodificación y Lectura de Registros (ID - Instruction Decode):** La unidad de control interpreta el código de operación (*opcode*). Se leen los valores almacenados en los registros fuente (rs, rt) dentro del banco de registros. Si la instrucción contiene un valor inmediato, este se extiende en signo a 32 bits.
3. **Ejecución / Cálculo de Dirección (EX - Execute):** La ALU realiza el cómputo principal. En operaciones Tipo R, ejecuta la operación matemática o lógica. En Tipo I de memoria, suma la dirección base con el desplazamiento (*offset*). En saltos condicionales, evalúa la comparación.
4. **Acceso a Memoria (MEM - Memory Access):** Interacción con la memoria principal de datos. Para lecturas (`lw`), se extrae el dato en la dirección calculada. Para escrituras (`sw`), el valor especificado se guarda en memoria. Para Tipo R, esta etapa permanece inactiva.

5. **Escritura de Resultados (WB - Write Back):** El valor procesado (de la ALU o de la memoria durante `lw`) se escribe permanentemente en el registro destino correspondiente (rd o rt).

7. Predominio de Tipos de Instrucciones (R, I, J)

Cuadro 4: Distribución estimada de tipos de instrucciones

Algoritmo	Tipo R	Tipo I	Tipo J
Mergesort	35 %	55 %	10 %
Quicksort	45 %	50 %	5 %

- **Instrucciones Tipo I (Predominantes):** Ambas implementaciones manejan el stack en MIPS32 mediante desplazamientos inmediatos (`addi $sp, $sp, -20`, `sw $ra, 16($sp)`), calculan direcciones sumando inmediatos o constantes, y ejecutan la mayoría de saltos condicionales de control de flujo (`bge`, `bgt`, `ble`). En Mergesort se observa una fuerte presencia de lecturas y escrituras directas (`lw $t4, 0($t3)`, `sw $t5, 0($t3)`).
- **Instrucciones Tipo R:** Se emplean para operaciones aritmético-lógicas entre registros (`add`, `sll`, `move`) y para el retorno explícito de funciones (`jr $ra`).
- **Instrucciones Tipo J:** Representan la proporción menor. Se reducen a saltos incondicionales absolutos dentro de los bucles de iteración (`j`) y a la invocación de procedimientos (`jal`).

8. Impacto del Abuso de Instrucciones de Salto

- **Penalizaciones por Salto (Branch Penalty):** La arquitectura MIPS32 utiliza un pipeline de 5 etapas. Si el procesador predice incorrectamente si el salto se tomará o no, las instrucciones leídas de forma especulativa deben descartarse (*flush* del pipeline), lo que desperdicia ciclos de reloj.
- **Desperdicio en la Ranura de Retardo (Branch Delay Slot):** MIPS32 ejecuta por diseño la instrucción que se encuentra inmediatamente después de un salto. Si no es posible reordenar el código con una instrucción útil, es necesario insertar instrucciones nulas (`nop`), aumentando el tamaño del código y consumiendo tiempo de CPU.
- **Pérdida de Localidad de Código y Misses de Caché:** Los saltos constantes a etiquetas distantes rompen la localidad espacial del código, aumentando la tasa de fallos (*cache misses*) en la I-Cache y obligando a acceder a la memoria principal.

9. Ventajas del Modelo RISC de MIPS

- **Conjunto de Instrucciones Reducido y Formato Fijo:** Todas las instrucciones tienen un tamaño uniforme de 32 bits, lo que simplifica la unidad de decodificación y permite que las operaciones fundamentales se ejecuten a alta velocidad en muy pocos ciclos.
- **Arquitectura Load/Store y Banco de Registros Amplio:** MIPS32 obliga a procesar datos únicamente dentro de su banco de 32 registros generales. En Quicksort, esto permite comparar e intercambiar elementos en registros temporales sin saturar la memoria.
- **Segmentación de Tuberías (Pipelining) Altamente Eficiente:** El diseño uniforme de 5 etapas maximiza el rendimiento (*throughput*) en bucles repetitivos de ordenamiento.

10. Uso del Modo Paso a Paso (Step / Step Into) en MARS

- **Uso del botón Step (F7):** Permitió avanzar la ejecución instrucción por instrucción dentro del mismo nivel de contexto. Se empleó en rutinas iterativas (`loop_comparar`, `part_loop`) para verificar en tiempo real cómo progresaban los índices (i, j, k) y si las condiciones de salida actuaban en el momento preciso.
- **Uso del botón Step Into:** Permitió .[en]trar.[en] el cuerpo de las funciones recursivas tras un `jal` para supervisar el prólogo del stack, confirmando la correcta reserva de espacio (`addi $sp, $sp, -20`) y el respaldo de `$ra` y `$s0`.
- **Validación de la Recursión:** Permitió observar la acumulación progresiva de marcos en la pila durante el proceso de división y verificar el proceso de retorno (epílogo) sin desbordar la pila.

11. Herramienta de MARS para Inspección de Registros

La herramienta más útil fue la tabla de registros (**Registers**), ubicada en el panel lateral derecho del simulador:

- **Monitoreo en Tiempo Real:** Muestra el valor en hexadecimal y decimal de los 32 registros, permitiendo supervisar la evolución de los índices temporales (`$t0`–`$t6`) y los punteros de memoria.
- **Detección Eficiente de Errores:** Facilitó verificar que los registros guardados (`$s0`) y `$ra` conservaran sus datos tras cada llamada recursiva, y confirmó la entrega del valor de retorno en `$v0`.
- **Eliminación de Overhead:** Evitó la necesidad de insertar múltiples llamadas al sistema (`syscall`) para imprimir datos intermedios, manteniendo el código limpio.

12. Visualización del Camino de Datos para Instrucción Tipo R (add)

Mediante la herramienta integrada **Tools** → **MIPS X-Ray**, para una instrucción Tipo R (como `add $s0, $a1, $a2`):

1. **IF:** La instrucción es leída desde la memoria de instrucciones y el PC se incrementa ($PC + 4$).
2. **ID:** La unidad de control decodifica el *opcode* y lee los valores contenidos en los registros fuente (`$a1` y `$a2`) desde el banco de registros.
3. **EX:** La ALU procesa la suma utilizando ambos operandos.
4. **MEM:** Etapa inactiva. No se realiza ninguna interacción con la memoria de datos.
5. **WB:** El resultado generado por la ALU se escribe de forma definitiva en el registro destino (`$s0`).

13. Visualización del Camino de Datos para Instrucción Tipo I (lw)

Mediante **Tools** → **MIPS X-Ray**, para una instrucción Tipo I de carga de memoria (como `lw $t4, 0($t3)`):

1. **IF:** La instrucción se obtiene de la memoria apuntada por el PC y este se incrementa ($PC + 4$).
2. **ID:** Se decodifica el *opcode*, se identifica el registro base (`$t3`), el destino (`$t4`) y el valor inmediato (offset), el cual se extiende en signo a 32 bits.
3. **EX:** La ALU suma la dirección base con el inmediato extendido para calcular la dirección efectiva.
4. **MEM:** Se activa completamente. El procesador accede a la memoria de datos usando la dirección calculada y extrae la palabra de 32 bits.
5. **WB:** El dato leído de la memoria se escribe permanentemente en el registro destino (`$t4`).

14. Justificación de los Algoritmos Seleccionados

- **Relevancia Conceptual y Recursividad:** Quicksort y Mergesort requieren una gestión recursiva explícita, sirviendo para poner en práctica la construcción manual de marcos de pila (*stack frames*) y la preservación de registros según el MIPS ABI.

- **Análisis In-Place vs. Memoria Auxiliar:** Quicksort permite analizar un enfoque *in-place* mediante intercambios (*swaps*), mientras que Mergesort evalúa el costo de un búfer auxiliar en RAM (`temp`) y el tráfico masivo de instrucciones `lw`/`sw`.
- **Eficiencia Algorítmica:** Demuestra cómo la reducción de complejidad asintótica a $\mathcal{O}(n \log n)$ compensa el *overhead* derivado del manejo de la pila (`addi $sp, $sp, -20`).

15. Análisis y Discusión de los Resultados

El estudio demuestra que la eficiencia en MIPS32 depende del equilibrio entre la complejidad teórica y la interacción con el camino de datos y la memoria. Mergesort ofrece un tiempo $\mathcal{O}(n \log n)$ garantizado, pero su fase de mezcla en el búfer auxiliar sobrecarga el bus de datos con instrucciones de carga y almacenamiento. Quicksort optimiza los accesos a RAM al operar *in-place*, aunque está expuesto a peores casos $\mathcal{O}(n^2)$ según la elección del pivote.

Asimismo, la disciplina en el manejo de la pila (`addi $sp, $sp, -20`) resulta vital para prevenir corrupciones durante la recursión. Las herramientas de depuración de MARS (paso a paso, panel de registros y MIPS X-Ray) permitieron validar el camino de datos (IF-ID-EX-MEM-WB) y confirmaron que la máxima optimización se logra combinando baja complejidad algorítmica con un uso intensivo del banco de registros internos.